

מבוא לתכנות מערכות – מבחן מועד א', סתיו תשע"ו

מרצה: ד"ר איתי שרון

משך המבחן: 180 דקות

חומר עזר מותר: 2 דפים המכילים תאור פקודות ב-C ו-bash. כל חומר עזר אחר אסור.

המבחן כולל 5 שאלות, יש לענות על כולן. בשאלות הדורשות נימוק הקפידו לספק אותו.

יש לענות על טופס הבחינה במקומות המיועדים לכך. ניתן להשתמש במחברת העזר אולם יש להגיש רק את טופס הבחינה.

העמוד האחרון במבחן כולל חומר שיכול (אך לא חייב!) לעזור לכם.

בהצלחה!

המכללה האקדמית תל חי, החוג למדעי המחשב

שאלה 1 (15 נקודות)

א. נתונה הפונקציה הבאה. תארו במילים מה היא עושה.

```
int annoying_function(const char* s, const char* w)
{
    int j, n;
    for(;*s; s++) {
        if(*s != *w)
            continue;
        for(j=0; w[j] && (s[j] == w[j]); j++)
            ;
        n += (w[j] == 0);
    }
    return n;
}
```

הפונקציה סופרת את מספר המופעים של w ב-s

ב. ממשו גרסא של הפקודה grep שמקבלת שם של קובץ ומילה כארגומנטים ומדפיסה את כל השורות המכילות את המילה. הניחו ששורה לא יכולה להיות ארוכה מ-1024 תווים. אין צורך לבדוק את תקינות הקלט.

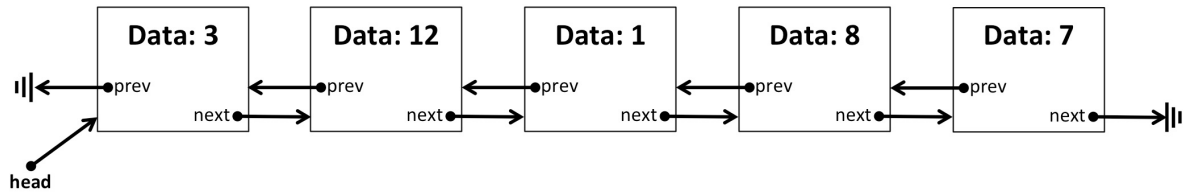
```
void grep(const char* file, const char* word)
```

```
{
    FILE* fp = fopen(file, "r");
    char line[1024];
    while(fgets(line, 1024, fp)) {
        if(strstr(line, word))
            printf("%s", line);
    }
}
```

המכללה האקדמית תל חי, החוג למדעי המחשב

שאלה 2 (30 נקודות)

רשימה מקושרת דו כיוונית (double linked list) היא רשימה מקושרת שבה כל אחד מהאיברים במערך מכיל מצביע לאיבר שבא אחריו וגם לזה שבא לפניו. ברשימה שלנו מותר לשמור בדיוק עותק אחד של כל אחד מהאיברים ברשימה. דוגמא לרשימה מקושרת דו-כיוונית ניתן למצוא באיור למטה. בשאלה זאת אתם נדרשים לממש באופן חלקי ADT של רשימה מקושרת דו-כיוונית.



א. כתבו header file עבור ADT של רשימה מקושרת דו כיוונית. על הרשימה להיות מסוגלת להכיל כל סוג של טיפוס (גנריות). הקובץ צריך להכיל הצהרות על הפונקציות הבאות ואחרות שאתם חושבים שנחוצות:

- פונקציית יצירת רשימה
- פונקציית הריסת רשימה
- הוספת איבר
- הסרת איבר
- מציאת איבר
- החזרת מספר האיברים ברשימה
- החזרת האיבר ה-n ברשימה

```
#ifndef DOUBLE_LINKED_LIST
```

```
#define DOUBLE_LINKED_LIST
```

```
typedef struct DoubleLinkedList *DoubleLinkedList;
```

```
DoubleLinkedList DLLCreate(bool (*gt)(void*, void*), void* (*cpy)(void*),  
void (*fre)(void*));
```

```
void DLLDestroy(DoubleLinkedList);
```

```
void DLLAdd(DoubleLinkedList, void*);
```

```
void DLLRemove(DoubleLinkedList, void*);
```

```
bool DLLFind(DoubleLinkedList, void*);
```

```
void DLLSize(DoubleLinkedList);
```

```
void DLLGetNth(DoubleLinkedList, int);
```

```
#endif
```

המכללה האקדמית תל חי, החוג למדעי המחשב

ב. כתבו את ה-structs (אחד או יותר) שבהם תעשו שימוש עבור הרשימה המקושרת הדו כיוונית. ממשו את פונקציית יצירת הרשימה.

```
struct DLLNode {  
    void*    data;  
  
    struct DLLNode*  prev;  
  
    struct DLLNode*  next;  
  
};  
  
typedef struct DLLNode* DLLNode;  
  
struct DoubleLinkedList {  
  
    bool (*gt)(void*, void*);  
  
    void* (*cpy)(void*);  
  
    void (*fre)(void*);  
  
    DLLNode head;  
  
};  
  
DoubleLinkedList DLLCreate(bool (*gt)(void*, void*), void* (*cpy)(void*),  
                            void (*fre)(void*))  
{  
  
    DoubleLinkedList dll = (DoubleLinkedList)malloc(sizeof(struct DoubleLinkedList));  
  
    dll->gt = gt;  
  
    dll->cpy = cpy;  
  
    dll->fre = fre;  
  
    dll->head = NULL;  
  
}
```

המכללה האקדמית תל חי, החוג למדעי המחשב

ג. ממשו את פונקצית הסרת האיבר מהרשימה.

```
void DLLRemove(DoubleLinkedList dll, void* item)
{
    DLLNode node;

    /* Start by looking for the node */
    for (node=dll->head;
         node && (dll->gt(node->data, item) || dll->gt(item, node->data));
         node=node->next)
        ;

    if (!node)
        return;

    if (node == dll->head) {
        dll->head = node->next;
        if (dll->head)
            dll->head->prev = NULL;
    }
    else {
        node->prev->next = node->next;
        if (node->next) {
            node->next->prev = node->prev;
        }
    }

    dll->fre(node->data);
    free(node);
}
```

המכללה האקדמית תל חי, החוג למדעי המחשב

שאלה 3 (20 נקודות)

בשאלה זו תממשו שתי פונקציות אשר מכווצות רצף של אפסים ואחדים לביטים. הקלט בשני המקרים הוא מערך של `char` והגודל שלו, כאשר המערך מכיל את ה**בתים** 0 או 1 בלבד. אין צורך לבדוק את נכונות הקלט. כמו כן אין צורך לבדוק שהקצאות זכרון הצליחו (במידה והקצתם).

א. כתבו פונקציה בשם `compress_uint` אשר מקבלת מערך של `char` המכיל אפסים ואחדים בלבד ואת הגודל שלו ומחזירה `unsigned int` שבו כל ביט מכיל את הערך של המקום המתאים במערך. כלומר: הביט ה-0 ב-`unsigned int` שיוחזר יכיל את הערך שבמקום ה-0 במערך, הביט במקום ה-1 יכיל את הערך במקום ה-1, וכו'. גודל המערך המועבר אינו גדול יותר מאשר `sizeof(unsigned int)*8` (אין צורך לבדוק את זה). במידה וגודל מערך הקלט אינו שווה ל-`sizeof(unsigned int)*8` מלאו את הביטים הנותרים ב-0.

```
typedef unsigned int uint;

uint compress_uint(char* array, int array_size) {

    uint res = 0;

    int i=0;

    for(i=0; i<array_size; i++) {

        char c = (array[i] == '1')? 1 : 0;

        res <<= 1;

        res |= c;

    }

    return res;

}
```

המכללה האקדמית תל חי, החוג למדעי המחשב

ב. כתבו פונקציה בשם `compress_array` אשר מקבלת מערך של `char` ואת הגודל שלו ומחזירה כתובת של מערך של `unsigned int` אשר מכיל את התוכן המכווץ של המערך המקורי. הביט ה-n במערך המוחזר צריך להכיל את הערך של המקום ה-n במערך המקורי. במידה ומערך הקלט אינו ממלא בדיוק את כל הביטים במערך התוצאה (כלומר: גודלו אינו מתחלק ב- $(\text{sizeof}(\text{unsigned int}) * 8)$) מלאו את הביטים הנותרים ב-0.

מותר (ואף רצוי!) להשתמש בפונקציה שכתבתם ב-א. מותר להשתמש בה גם אם לא מימשתם אותה.

```
uint* compress_array(char* buffer, int buffer_size) {  
    int i, compressed_buffer_size = buffer_size / (sizeof(uint)*8);  
    if(buffer_size%(sizeof(uint)*8) != 0)  
        compressed_buffer_size++;  
    uint* res = (uint*)malloc(sizeof(uint)*compressed_buffer_size);  
    for(i=0; i<compressed_buffer_size; i++) {  
        int n = (buffer_size-(i*sizeof(uint)*8) > sizeof(uint)*8)?  
                sizeof(uint)*8 : (buffer_size-(i*sizeof(uint)*8));  
        res[i] = compress_uint(buffer+(i*sizeof(uint)*8), n);  
    }  
    return res;  
}
```

המכללה האקדמית תל חי, החוג למדעי המחשב

שאלה 4 (20 נקודות)

שאלה זו עוסקת בעצי חיפוש בינאריים.

להלן ממשק של עץ חיפוש בינארי הדומה לזה שראינו בכתה.

```
struct BNode {
    struct BNode *left, *right;
    int data;
};

typedef struct BNode *BNode;

BNode BSTCreate();
BNode BSTAddNode(BNode, int);
BNode BSTRemoveNode(BNode, int);
bool BSTSearchNode(BNode, int);
void BSTinorder(BNode);
```

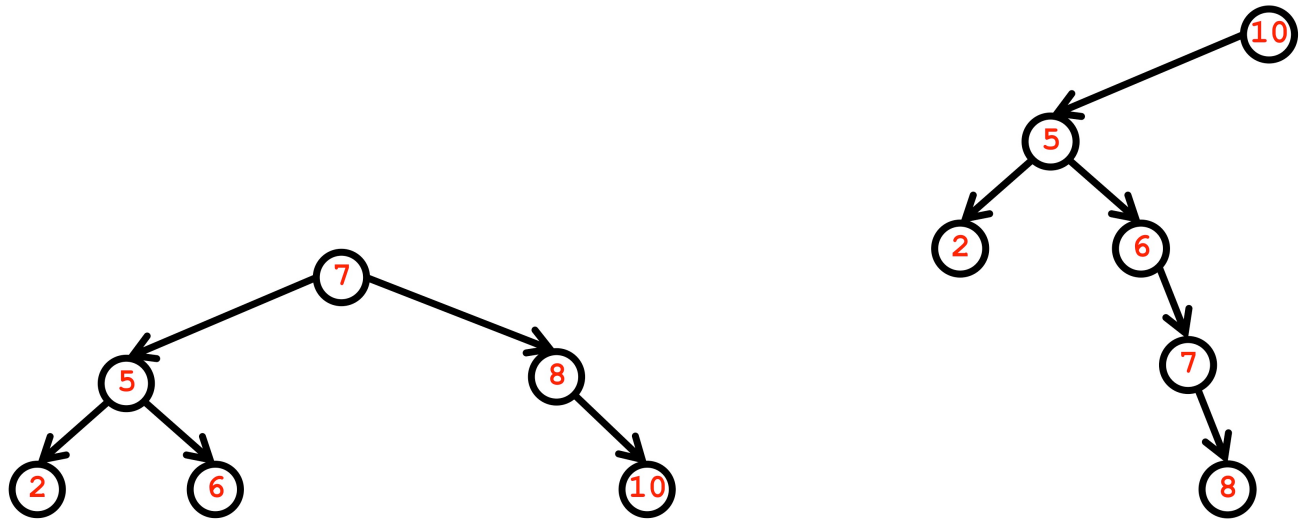
א. כתבו פונקציה אשר מקבלת שורש של עץ חיפוש בינארי ומחזירה את הבדלי העומקים בין שני הצדדים. הפונקציה תחזיר ערך קטן מ-0 במידה וצד שמאל עמוק מצד ימין, ערך גדול מ-0 אם צד ימין עמוק מצד שמאל או 0 אם העומק של שני הצדדים שווה.

למשל: עבור העצים הנתונים בעמוד הבא הפונקציה צריכה להחזיר 0 עבור הצומת 7 בעץ השמאלי ו-4 (מינוס ארבע) עבור הצומת 10 שבעץ הימני.

```
int depth difference(BNode* node) {
    if(node == NULL)
        return 0;
    return (depth(node->left) - depth(node->right));
}

int depth(BNode node)
{
    if(node == NULL)
        return -1;
    int l = depth(node->left)+1;
    int r = depth(node->right)+1;
    return (l>r? l : r);
}
```


המכללה האקדמית תל חי, החוג למדעי המחשב



ב. עצי חיפוש בינאריים יכולים להיות מאוזנים, במידה והעומק של שני צדי העץ זהים או נבדלים לכל היותר ב-1, או לא מאוזנים, במידה והבדלי העומקים גדולים מ-1. למשל: בדוגמא למעלה העץ השמאלי הוא מאוזן והעץ הימני לא. עצים מאוזנים נחשבים לעדיפים מכיוון שפעולות הכנסה, חיפוש והוצאה הן מהירות יותר עליהם.

כתבו פונקציה שבונה עץ חיפוש בינארי מאוזן בהנתן מערך של מספרים ממויינים.

```
BNode BSTCreateBalanced(char* array, int array_size) {  
    BNode node = BSTCreate();  
    recursive_add_element(node, array, array_size);  
}  
  
void recursive_add_element(BNode node, char* array, int array_size) {  
    if(array_size <= 0)  
        return;  
    BSTAddNode(node, array[array_size/2]);  
    int size_left = (array_size%2 == 0)? (array_size/2-1) : (array_size/2);  
    recursive_add_element(node, array, size_left);  
    recursive_add_element(node, array+array_size/2+1, array_size-size_left-1);  
}
```

המכללה האקדמית תל חי, החוג למדעי המחשב

שאלה 5 (15 נקודות)

כתבו סקריפט bash אשר בדומה ל-egrep מקבל שם של קובץ ו-regular expression ומחזיר את כל השורות בקובץ שמתאימות ל-regular expression. אין להשתמש בפקודות grep או egrep לצורך כתיבת הסקריפט.

למשל: עבור קובץ המכיל את הטקסט הבא:

```
Mary had a little lamb
Little lamb, little lamb
Mary had a little lamb
Its fleece was white as snow
And everywhere that Mary went
Mary went, Mary went
Everywhere that Mary went
The lamb was sure to go
```

וה-regular expression `M. ry` הסקריפט יחזיר את השורות הבאות:

```
itaish@cs ~ $ ./my_egrep Mary.txt M.ry
Mary had a little lamb
Mary had a little lamb
And everywhere that Mary went
Mary went, Mary went
Everywhere that Mary went
```

אילו regular expressions תצטרכו להעביר לסקריפט על מנת שהוא יחזיר את

- כל השורות אשר מתחילות באות שאינה M? הניחו ששורות מתחילות באותיות גדולות או מספרים.
- כל השורות המסתיימות במילים בנות 4 אותיות לפחות?
- כל השורות המכילות מילים עם שני e או יותר?

ביטויים:

כל השורות אשר מתחילות באות שאינה M: `^[^M]`

כל השורות המסתיימות במילים בנות 4 אותיות לפחות: `[A-Za-z]{4}$`

כל השורות המכילות מילים עם שני e או יותר: `e.*e`

הסקריפט בעמוד הבא.

המכללה האקדמית תל חי, החוג למדעי המחשב

```
#!/bin/bash
```

```
if [ $# -ne 2 ]
```

```
then
```

```
    echo -e "\nUsage: $0 <file> <pattern>\n"
```

```
else
```

```
    while read line
```

```
    do
```

```
        if [[ $line =~ $2 ]]
```

```
        then
```

```
            echo $line
```

```
        fi
```

```
    done < "${1}"
```

```
fi
```

המכללה האקדמית תל חי, החוג למדעי המחשב

פקודות bash

- cat – מדפיסה תוכן של קובץ למסך
- head – מדפיסה את 10 השורות הראשונות של קובץ למסך (ניתן לשנות את המספר ע"י שימוש ב -n)
- tail – מדפיסה את 10 השורות האחרונות של קובץ למסך (ניתן לשנות את המספר ע"י שימוש ב -n)
- ls – מדפיסה תוכן ספרייה, או רשימה של קבצים אשר מתאימה לתאור שהעביר המשתמש כארגומנט (למשל: ls *.c תדפיס את שמות הקבצים שמסתיימים ב-c. בספרייה הנוכחית)
- sort – ממיינת לקסיקוגרפית קובץ קלט. ניתן למיין מספרית (-n) ולשמור רק עותק אחד של שורות שחוזרות על עצמן (-u)
- wc – מחזירה את מספר השורות (-l), בתים (-c) או מילים (-w) בקובץ
- read – קוראת שורה מה-standard input ומכניסה אותה למשתנה שמועבר ע"י המשתמש כארגומנט
- find – מחזירה רשימה של כל הקבצים אשר מתאימים לתאור המשתמש (ארגומנט -name) בספרייה מסוימת ואלו שנמצאות תחתיה (ארגומנט ראשון לפקודה)
- grep – מחזירה שורות אשר מכילות string מסויים. ניתן להשתמש ב-regular expressions (-e), ולהחזיר שורות שאינן מכילות את ה-string אותו מחפשים (-v)
- egrep – זהה ל-grep -e
- cut – מחזירה שדות מסויימים בכל שורה (-f) בהתאם ל-delimiter (-d). ניתן גם להחזיר בתים מסויימים בכל שורה (-b)

פעולות על ביטים

- bitwise AND - &
- bitwise OR - |
- bitwise XOR - ^
- right shift - >>
- left shift - <<
- One's complement - ~

פונקציות C-ב

- int putchar(int character, FILE* fp) – כותבת את character ל-fp.
- int getc(FILE* fp) – קוראת תו אחד מתוך fp.
- scanf(const char* format,...) – קוראת מידע מתוך stdin בהתאם ל-format, מאחסנת את המידע בכתובות המשתנים שמסופקים. fscanf(FILE* stream, const char* format,...) – אותו דבר כאשר הקלט נקרא מתוך stream.
- sscanf(const char* s, const char* format,...) – אותו הדבר כאשר הקלט נקרא מתוך s.
- printf(const char* format,...) – כתיבת מידע ערוך ל-stdout בהתאם ל-format והמשתנים שמסופקים.
- fprintf(FILE* stream, const char* format,...) – אותו דבר כאשר הכתיבה נעשית לתוך stream.
- sprintf(const char* s, const char* format,...) – אותו הדבר כאשר הכתיבה נעשית לתוך s.
- void* malloc(size_t size) – מקצה זכרון בגודל size
- void* calloc(size_t num, size_t size) – מקצה num יחידות בגודל size כל אחת ומאפסת את כל הזכרון שהוקצה.
- void* realloc(void* ptr, size_t size) – מקצה size בתים, מוסיפה/מורידה מכמות הזכרון ש-ptr מצביעה עליו (צריך להיות מוקצה דינמית גם כן), עשויה לשנות את הזכרון המוקצה ואז תעתיק את התוכן של ptr לשטח החדש. מחזירה את כתובת הזכרון שהוקצה.
- const char* strstr(const char* str1, const char* str2) – מחזירה את המקום שבו מופיעה str2 ב-str1 (או NULL אם היא לא מופיעה)
- int strcmp(const char* str1, const char* str2) – משווה את str1 ל-str2, מחזירה 0 אם הן זהות, ערך קטן מ-0 אם str1 קטן לקסיקוגרפית מ-str2 או ערך גדול מ-0 אחרת.
- size_t strlen(const char* str) – מחזירה אורך של מחרוזת.